Cómputo paralelo híbrido (MPI + OpenMP) para un motor Monte Carlo en un clúster de PCs de laboratorio: caso Batalla Naval como plataforma didáctica en Yacuiba, Chaco boliviano

Fernando Daniel Cruz Zarate
Carrera de Ingeniería Informática, Universidad Autónoma Juan Misael Saracho (UAJMS)
Yacuiba, Tarija, Bolivia
[correo electrónico]

***Resumen*—Se presenta el diseño, la implementación y la evaluación de un motor de simulación Monte Carlo paralelo, desarrollado como núcleo de la inteligencia artificial "Experta" de un juego de Batalla Naval, y su despliegue sobre un clúster de [n] PCs de laboratorio. El trabajo se sitúa en Yacuiba (Chaco boliviano) y propone aprovechar el parque de PCs de escritorio de la universidad como clúster de bajo costo para la enseñanza práctica de cómputo paralelo y como plataforma de simulación. El motor se implementó en tres capas que producen resultados idénticos bit a bit: Node.js (`worker_threads`), C con OpenMP y C con MPI + OpenMP (modelo híbrido). El particionamiento es por trozos independientes con semillas derivadas, seguido de una reducción colectiva. Con [p] unidades de cómputo se midió una aceleración de [S]× (eficiencia de [E] %), una fracción serial estimada de [f] (Karp–Flatt) y un límite de Amdahl de [1/f]×.**

***Abstract*—This paper presents the design, implementation and evaluation of a parallel Monte Carlo simulation engine, built as the core of the "Expert" AI of a Battleship game and deployed on a cluster of [n] laboratory PCs located in Yacuiba (Bolivian Chaco). The engine is implemented in three layers that yield bit-identical results: Node.js worker threads, C with OpenMP, and C with hybrid MPI + OpenMP. Work is partitioned into independent chunks with derived seeds and combined by a collective reduction. With [p] processing units, a speedup of [S]× (efficiency [E] %) was measured, with an estimated serial fraction of [f] (Karp–Flatt) and an Amdahl limit of [1/f]×.**

***Index Terms*—cómputo paralelo, MPI, OpenMP, Monte Carlo, clúster, speedup, ley de Amdahl, Chaco boliviano.**

I. Introducción y contextualización regional

Yacuiba es capital de la provincia Gran Chaco del departamento de Tarija, se encuentra a unos 3 km de la frontera con Argentina y forma conurbación con Profesor Salvador Mazza; su economía se apoya en el comercio fronterizo y en los recursos hidrocarburíferos [1]. El Censo 2012 registró 91 998 habitantes en la ciudad [1] y 147 478 en la provincia, con un crecimiento provincial de 26,79 % entre 2001 y 2012 [2]. La UAJMS mantiene en Yacuiba la Facultad de Ingeniería de Recursos Naturales y Tecnología, donde se dicta Ingeniería Informática [3], y en la región se investigan soluciones tecnológicas para la educación digital y la automatización agrícola [4].

Problema. La formación en cómputo paralelo requiere practicar con equipos multinúcleo y con clústeres, cuyo costo puede ser una barrera cuando no se dispone de infraestructura dedicada. Un laboratorio universitario, en cambio, ya cuenta con PCs de escritorio (Dell OptiPlex) que, interconectadas por una red conmutada, pueden operar como un clúster de bajo costo del tipo Beowulf [5]. Falta, sin embargo, una aplicación concreta, reproducible y medible que sirva a la vez como material didáctico y como banco de pruebas de simulación. **Propuesta:** un motor Monte Carlo (estimación de probabilidades por simulación, técnica base del análisis de riesgo [6]) integrado a un juego de Batalla Naval que los estudiantes pueden ejecutar, medir y extender.

Alcance y limitaciones. Este trabajo es un prototipo académico: no se afirma su adopción por ninguna institución **[si existe constancia o acuerdo institucional, citarlo aquí]**. Las aplicaciones productivas (por ejemplo, simulación de riesgo en actividades locales) se presentan como línea de trabajo futura, no como resultados.

Contribuciones. (i) Un núcleo Monte Carlo con tres implementaciones cuyo resultado es idéntico para los mismos parámetros, lo que permite verificar su corrección y comparar modelos de programación; (ii) una arquitectura de dos capas (cómputo en hardware directo; aplicación en contenedores); (iii) una metodología de medición con speedup, eficiencia, Karp–Flatt y escalabilidad fuerte y débil; (iv) el código y los scripts para reproducir todo el estudio.

II. Marco teórico y estado del arte

II-A. Taxonomía de Flynn

Flynn clasifica las arquitecturas según el número de flujos de instrucciones y de datos: SISD, SIMD, MISD y MIMD [7]. El sistema desarrollado es **MIMD** en su modalidad SPMD: cada hilo o proceso ejecuta el mismo programa, con su propio flujo de control, sobre un trozo distinto de las muestras. Dentro de cada hilo el cálculo es secuencial (SISD); no se explota SIMD explícito, de modo que el paralelismo proviene únicamente de la descomposición de datos entre flujos independientes.

Tabla I
CLASIFICACIÓN DE LOS COMPONENTES DEL SISTEMA.

Componente	Flynn	Memoria	Comunicación
Node.js, `worker_threads`	MIMD (SPMD)	compartida o privada	`Atomic`
C + OpenMP	MIMD (SPMD)	compartida	variables
C + MPI + OpenMP	MIMD (SPMD)	híbrida	MPI sob
Nodos de IA (Docker)	MIMD (maestro–trabajadores)	distribuida	HTTP/TC

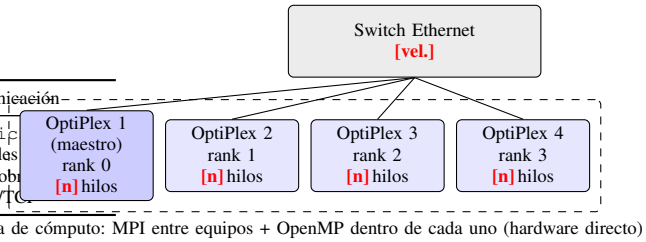


Figura 1. Arquitectura del clúster. El número de equipos, hilos por equipo y la velocidad de la red se completan con el hardware real.

II-B. Modelos de memoria

En **memoria compartida** los hilos acceden al mismo espacio de direcciones y se coordinan con barreras o operaciones atómicas; OpenMP es el estándar de facto en C [8], [9], y Node.js ofrece un equivalente con `SharedArrayBuffer` y `Atomics` [10]. En **memoria distribuida** cada proceso tiene su espacio privado y la cooperación se hace por paso de mensajes, cuyo estándar es MPI [11]. Los PCs de laboratorio son monoprocesador de acceso uniforme a memoria; un clúster de ellos es entonces un sistema **híbrido**: memoria compartida dentro de cada equipo y distribuida entre equipos, lo que motiva el modelo MPI + OpenMP [12], [13].

II-C. Redes de interconexión

Un clúster de PCs se conecta con Ethernet conmutada: topología en **estrella** a través de un switch, con diámetro 2 y una capacidad de bisección limitada por la velocidad del enlace de cada equipo [13]. El coste de enviar m bytes se modela con la relación de Hockney [14]

$$T_{\text{msg}}(m) = \alpha + \frac{m}{\beta}, \tag{1}$$

con latencia α y ancho de banda β. Una reducción colectiva en árbol cuesta aproximadamente $\lceil \log_2 p \rceil \, T_{\text{msg}}(m)$. En este trabajo $m = 4n^2$ bytes (un entero por casilla; 1 600 B para $n = 20$), *independiente del número de muestras*: se predice por tanto que la comunicación es despreciable frente al cómputo cuando N es grande, hipótesis que la sección V contrasta.

II-D. Clasificación del sistema

La Tabla I resume cómo se clasifica cada componente según Flynn, el modelo de memoria y el medio de comunicación.

II-E. Métricas y leyes de rendimiento

Con T_1 el tiempo de la mejor versión secuencial y T_p el tiempo con p unidades de cómputo: aceleración $S = T_1/T_p$ y eficiencia $E = S/p$. La ley de Amdahl [15] acota el *mismo* trabajo con una fracción secuencial f:

$$S(p) = \frac{1}{f + \frac{1-f}{p}} \le \frac{1}{f}. \tag{2}$$

La de Gustafson [16] considera que el trabajo crece con p (escalabilidad débil), con fracción secuencial escalada α_G:

$$S_G(p) = p - \alpha_G \, (p - 1). \tag{3}$$

La métrica de Karp–Flatt [17] estima f a partir de las mediciones: $e = \dfrac{1/S - 1/p}{1 - 1/p}$. Si e es aproximadamente constante al crecer p, la limitación es una fracción serial fija; si crece, hay sobrecarga que aumenta con p (comunicación, contención, desbalance).

II-F. Método Monte Carlo

Dado el estado de un tablero (aguas, impactos, barcos restantes), se generan N flotas aleatorias compatibles; la probabilidad de que la casilla i contenga un barco se estima como $\hat{p}_i = \frac{1}{N_v} \sum_s \mathbf{1}[i \in F_s]$, con $N_v \le N$ las muestras válidas. El error típico decrece como $O(1/\sqrt{N_v})$ [6], por lo que reducirlo a la mitad cuesta cuadruplicar el cómputo: una necesidad real de paralelismo. Las muestras son independientes, lo que hace al problema *embarazosamente paralelo*: solo la reducción final requiere comunicación.

III. DISEÑO DE LA ARQUITECTURA E INFRAESTRUCTURA

III-A. Visión general

La arquitectura (Fig. 1) tiene dos capas. La **capa de cómputo** ejecuta el núcleo MPI + OpenMP directamente sobre el sistema operativo de los equipos, para que la medición no se vea afectada por la virtualización. La **capa de aplicación** (juego, base de datos y nodos de IA) corre en contenedores y delega el cálculo pesado a nodos de IA por HTTP.

III-B. Clúster físico y asignación de CPU y RAM

La Tabla II resume los equipos. Se asigna **un proceso MPI por equipo** y tantos hilos OpenMP como núcleos físicos: así la red solo transporta un mensaje por equipo en la reducción y la memoria compartida se aprovecha localmente. La memoria no es el factor limitante: el pico de memoria residente medido para un proceso en un tablero de 20×20 fue de 10,8 MB (entorno de desarrollo), muy inferior a la RAM de cualquier OptiPlex.

III-C. Configuración de contenedores

La aplicación se despliega con `docker compose`: servicios `db` (PostgreSQL), `backend`, `frontend` y, con el perfil `distributed`, `ai-node` escalable (`-scale ai-node=N`). Los nodos de IA no publican puertos (solo son alcanzables por la red interna), exigen un token compartido,

Tabla II
EQUIPOS DEL CLÚSTER (COMPLETAR CON `LSCPU`, `FREE -H`, `ETHTOOL`).

Equipo	CPU	Núc./hilos	RAM	Rol
OptiPlex 1	[modelo]	[n/h]	[GB]	maestro, rank 0, Docker
OptiPlex 2	[modelo]	[n/h]	[GB]	rank 1
OptiPlex 3	[modelo]	[n/h]	[GB]	rank 2
OptiPlex 4	[modelo]	[n/h]	[GB]	rank 3

Red: switch [marca/modelo], enlaces [Mb/s]; SO [distribución]; gcc [ver.]; OpenMPI [ver.]; Node.js 22.

validan y acotan cada campo recibido, y se limitan en CPU y memoria con `deploy.resources.limits`; si ningún nodo responde, el backend calcula con sus propios hilos (respaldo local).

```
docker compose --profile distributed \
  up --build --scale ai-node=3
# .env: AI_NODES=http://<nodo>:5001,...
# AI_NODE_TOKEN=<secreto>
```

IV. IMPLEMENTACIÓN ALGORÍTMICA PARALELA

IV-A. *Algoritmo y particionamiento de datos*

Para cada muestra se colocan los barcos restantes, de mayor a menor, eligiendo al azar una posición legal que no solape a las ya puestas; si quedan impactos sin cubrir se sesga la elección hacia posiciones que los cubran (90 %), y la muestra se descarta si no los cubre todos. Las casillas ocupadas acumulan un contador. Las N muestras se parten en $P = r \cdot t$ **trozos** (r procesos MPI, t hilos por proceso); el trozo k recibe $\lfloor N/P \rfloor + [k < N \bmod P]$ muestras y la semilla $\text{seed} + (k+1) \cdot \texttt{0x9E3779B1}$. Como el trozo k es el mismo cualquiera sea el reparto en procesos e hilos, *el resultado depende solo de* (N, P, seed) *y no de la topología*: esto es lo que permite verificar la corrección comparando una suma de control.

IV-B. *OpenMP: hilos dentro de un equipo*

Cada trozo es una iteración de un `parallel for`; cada hilo acumula en un arreglo local y lo vuelca una sola vez al arreglo compartido con `atomic`, de modo que hay pocas escrituras compartidas y se evita la contención y el *false sharing*.

```
#pragma omp parallel for schedule(static,1) \
        num_threads(T) reduction(+:total_acc)
for (int i = 0; i < count; i++) {
  int k = first + i; /* trozo global */
  sample_density(..., chunk(k), chunk_seed(seed,k), local,
        &acc);
  for (int j = 0; j < n; j++) if (local[j]) {
    #pragma omp atomic
    density[j] += local[j];
  }
  total_acc += acc;
}
```

IV-C. *MPI: procesos entre equipos*

`MPI_Init_thread` con `MPI_THREAD_FUNNELED` (solo el hilo principal invoca MPI); `MPI_Comm_rank/size` para ubicar el proceso; el proceso r calcula los trozos $[rt, (r+1)t)$; `MPI_Barrier` y `MPI_Wtime` delimitan la medición; `MPI_Reduce` (`MPI_SUM`) suma las densidades parciales y las muestras válidas en el proceso 0; reducciones con `MPI_MAX` y `MPI_SUM` de los tiempos dan el cómputo del proceso más lento y el desbalance máx/media; y `MPI_Gather` recoge los nombres de equipo para contar los nodos físicos. Se miden por separado el cómputo y la comunicación (la reducción), con $T_p = T_{\text{cómputo}} + T_{\text{com}}$.

Tabla III
ESCALABILIDAD FUERTE (MISMO TRABAJO): TIEMPO, ACELERACIÓN, EFICIENCIA, PORCENTAJE DE COMUNICACIÓN Y DESBALANCE.

Config.	p	T_p (ms)	S	E (%)	Com. (%)	Desb.
[pendiente de medición]						

IV-D. *Balanceo de carga y tolerancia a fallos*

En el núcleo en C el reparto es **estático**, adecuado porque los trozos tienen igual número de muestras independientes y el clúster es homogéneo; el desbalance medido (máx/media del cómputo) lo cuantifica. En la capa de aplicación (Node.js) el balanceo es **dinámico**: los hilos toman tareas de una cola compartida apenas quedan libres, con dos prioridades (partidas en vivo antes que torneos), y el coordinador asigna cada tarea al nodo de menor carga relativa (tareas en vuelo entre hilos). Si un nodo falla se le marca caído unos segundos y la tarea se reintenta en otro; es seguro porque las tareas son *idempotentes* (la semilla fija el resultado). Con cola larga la IA reduce sus muestras (contrapresión) y las tareas se cancelan cuando el jugador abandona la sala.

IV-E. *Verificación de corrección*

Para varios tableros (10, 12 y 20), tamaños de muestra y números de trozos (1 a 5), las versiones Node.js, C + OpenMP y C + MPI + OpenMP (p. ej. 2 procesos $\times$ 2 hilos y 3 procesos $\times$ 1 hilo) produjeron el mismo número de muestras válidas y la misma suma de control de la densidad. El repositorio incluye pruebas automatizadas (93 en el backend) que verifican, además, que los modos de comunicación por mensajes y por memoria compartida coinciden y que el resultado distribuido por red coincide con el local.

V. RESULTADOS EXPERIMENTALES Y ANÁLISIS DE RENDIMIENTO

V-A. *Metodología*

Escena de referencia: tablero de [n] $\times$ [n] con dos impactos adyacentes y cuatro casillas de agua, $N =$ [N] muestras. Cada punto es la **mediana** de varias repeticiones tras una de calentamiento; T_1 es la ejecución de 1 proceso $\times$ 1 hilo ([T1] ms). Una configuración es *equipos/procesos/hilos* y $p = r \cdot t$. Los equipos no ejecutaron otras tareas durante la medición. La red se caracterizó con un ping-pong MPI: $\alpha =$ [α] μs y $\beta =$ [β] MB/s (Ec. 1).

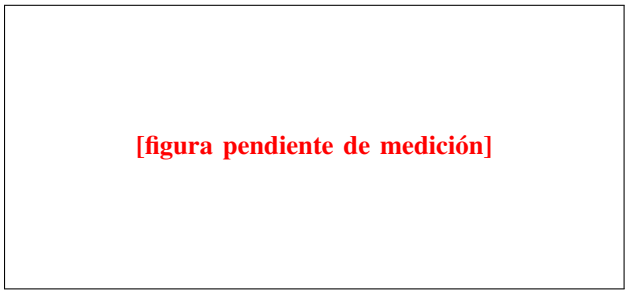


Figura 2. Aceleración medida frente al ideal y al modelo de Amdahl (Ec. 2) con f estimada por Karp–Flatt.

Tabla IV
ESCALABILIDAD DÉBIL (TRABAJO PROPORCIONAL A p). $S_{esc} = pT_1/T_p$; S_G ES EL MODELO DE LA EC. 3.

Config.	p	N	T_p (ms)	E_d (%)	S_{esc}	S_G
[pendiente de medición]						

V-B. *Escalabilidad fuerte*

V-C. *Escalabilidad débil y Gustafson*

V-D. *Validación de las leyes de Amdahl y Gustafson*

La fracción secuencial estimada con Karp–Flatt es $f =$ **[f]**, que por la Ec. 2 implica una aceleración máxima de **[1/f]**$\times$ para este trabajo; la mayor medida fue **[S]**$\times$ con $p =$ **[p]**. En escalabilidad débil, la fracción secuencial escalada es $\alpha_G =$ **[α_G]** y la aceleración escalada máxima, **[S_G]**$\times$ (Ec. 3). **[Comparar: ¿el modelo de Amdahl reproduce la curva medida? ¿e es constante o crece con p? Explicar las desviaciones.]**

V-E. *Análisis de cuellos de botella*

Comunicación. El mensaje de la reducción mide $4n^2$ bytes, independiente de N; la comunicación representó como máximo **[%]** % del tiempo total. **[Contrastar con el modelo $\lceil \log_2 p \rceil(\alpha + m/\beta)$ y explicar si crece con el número de equipos.]** **Desbalance.** El cociente máximo/media del cómputo fue como máximo **[desb.]**. **[Explicar causas: procesos del sistema, frecuencia de CPU, equipos distintos.]** **Memoria.** El pico residente (10,8 MB por proceso) indica que el problema está limitado por cómputo y por la caché, no por la capacidad de RAM. **[Si se mide p mayor que los núcleos físicos, discutir el efecto de hyper-threading y del ancho de banda de memoria.]**

V-F. *Capa de aplicación (Node.js)*

Con `worker_threads` la mejor aceleración fue **[S]**$\times$ con **[p]** hilos. Las tareas muy pequeñas (IA "difícil", microsegundos) no se benefician de repartirse: el experimento de granularidad lo comprueba y el sistema las calcula en el hilo principal. **[Resumir `npm run latency` (retraso del event loop) y `npm run load` (partidas por segundo) si se miden.]**

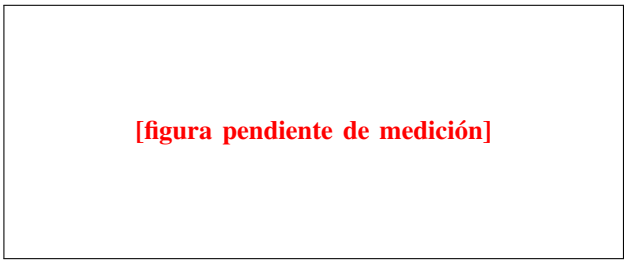


Figura 3. Eficiencia en escalabilidad fuerte y débil.

VI. CONCLUSIONES E IMPACTO SOCIAL Y PRODUCTIVO

Se diseñó e implementó un motor Monte Carlo paralelo con tres implementaciones verificadas entre sí y una arquitectura de dos capas reproducible sobre PCs de laboratorio. La descomposición en trozos independientes con semillas derivadas hace que el resultado no dependa de la topología, lo que simplifica la validación y la enseñanza. **[Concluir con los números medidos: aceleración, eficiencia, validez de Amdahl/Gustafson y cuello de botella dominante.]**

Impacto social. El proyecto ofrece a la carrera de Ingeniería Informática un material práctico y de código abierto para enseñar y medir cómputo paralelo con equipos que la universidad ya posee, y un banco de pruebas para trabajos de estudiantes. **Impacto productivo (potencial).** La simulación Monte Carlo es una técnica general de análisis de riesgo y estimación; un clúster local permitiría resolver simulaciones de ese tipo sin depender de infraestructura externa. Esta aplicación no se ha evaluado en este trabajo.

Limitaciones y trabajo futuro. La IA "Experta" no supera en calidad de juego a la "Difícil": su valor es ser una carga pesada y divisible para estudiar el paralelismo. Quedan pendientes la distribución de las salas de juego entre varias instancias (Redis), la comparación con MPI en más nodos y una carga de trabajo directamente regional con datos reales.

REFERENCIAS

[1] Banco Central de Bolivia, "Memoria del VIII concurso de ensayo escolar," 2015, [Online]. Available: https://www.bcb.gob.bo/webdocs/concurso/VIII/memoria_2015.pdf.

[2] Instituto Nacional de Estadística (Bolivia), "La población de la provincia Gran Chaco creció en 26,79 % entre los censos 2001 y 2012," Nota de prensa, Tarija, Bolivia, Aug. 2014, [Online]. Available: https://anda4.ine.gob.bo/ANDA4_2/index.php/catalog/184/download/895.

[3] Facultad de Ingeniería de Recursos Naturales y Tecnología, UAJMS, "Revista científica IYARAKUAA – dueño del conocimiento," *Rev. Cient. IYARAKUAA*, vol. 5, no. 14, Oct. 2024, iSSN 2707-4064. [Online]. Available: https://dicyt.uajms.edu.bo/revistas/index.php/iyarakua.

[4] O. A. Choque Gonzales, "Investigación de soluciones tecnológicas innovadoras para impulsar la educación digital, automatización agrícola y preservación cultural en el Chaco boliviano," *Rev. Cient. IYARAKUAA*, 2024, iSSN 2707-4064. [Online]. Available: https://dicyt.uajms.edu.bo/revistas/index.php/iyarakua. Verificar volumen y año.

[5] T. Sterling, D. J. Becker, D. Savarese, J. E. Dorband, U. A. Ranawake, and C. V. Packer, "BEOWULF: A parallel workstation for scientific computation," in *Proc. 24th Int. Conf. Parallel Process.*, vol. I, 1995, pp. 11–14.

[6] N. Metropolis and S. Ulam, "The Monte Carlo method," *J. Amer. Statist. Assoc.*, vol. 44, no. 247, pp. 335–341, 1949.

[7] M. J. Flynn, "Very high-speed computing systems," *Proc. IEEE*, vol. 54, no. 12, pp. 1901–1909, Dec. 1966.

[8] L. Dagum and R. Menon, “OpenMP: an industry standard API for shared-memory programming,” *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan. 1998.
[9] OpenMP Architecture Review Board, “OpenMP application programming interface, version 5.2,” Tech. Rep., Nov. 2021, [Online]. Available: https://www.openmp.org/specifications/.
[10] OpenJS Foundation, “Node.js documentation: Worker threads,” [Online]. Available: https://nodejs.org/api/worker_threads.html. Consultado: oct. 2026.
[11] Message Passing Interface Forum, “MPI: A message-passing interface standard, version 4.0,” Tech. Rep., Jun. 2021, [Online]. Available: https://www.mpi-forum.org/docs/.
[12] P. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA, USA: Morgan Kaufmann, 2011.
[13] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Harlow, U.K.: Addison-Wesley, 2003.
[14] R. W. Hockney, “The communication challenge for MPP: Intel Paragon and Meiko CS-2,” *Parallel Comput.*, vol. 20, no. 3, pp. 389–398, 1994.
[15] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.
[16] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988.
[17] A. H. Karp and H. P. Flatt, “Measuring parallel processor performance,” *Commun. ACM*, vol. 33, no. 5, pp. 539–543, May 1990.